

設定ファイル完全ガイド

CLAUDE.md / Rules / Skills / Agents / Settings / Hooks

Claude Codeのプロジェクト設定は**6種類のファイル・仕組み**で構成される。
CLAUDE.mdで基本行動を定義し、Rules・Skills・Agentsで機能拡張、Settings・Hooksで権限制御と自動化を行う。
すべてを組み合わせることで、確認なし・全自動の資料化エージェントを構築できる。

目次

1. CLAUDE.md — プロジェクトの「憲法」
2. Rules — ドメイン別ルール of 分割管理
3. Skills — カスタムスラッシュコマンド
4. Agents — 専門特化サブエージェント
5. Settings — 権限制御
6. Hooks — イベント駆動の自動処理

1 CLAUDE.md — プロジェクトの「憲法」

必須

Claudeが**セッション開始時に必ず読む**設定ファイル。ここに書いたことが全行動の前提になる。
複数の場所に置いて、全部あれば**全部読む**（上書きではなく追加）。

置き場所と読み込み優先順位

パス	適用範囲
<code>~/ .claude/CLAUDE.md</code>	グローバル（全プロジェクト共通）
<code>~/project/CLAUDE.md</code>	プロジェクトルート
<code>~/project/ .claude/CLAUDE.md</code>	.claudeフォルダ内

記入例 — 資料化PJのCLAUDE.MD

CLAUDE.md（記入例）

```
# プロジェクト：資料化PJ

## 役割
あなたはPDF・テキスト資料を読み込み、わかりやすく要約するエージェントです。

## 必須ルール
- 確認（「YESですか？」等）は一切不要。全て実行してよい
- 要約ファイルは必ず`要約資料/`フォルダに保存する
- ファイル名は内容を表す日本語で命名する（例：差別化物販戦略略塾_要約.md）

## 要約フォーマット
### 必ず含める項目
1. 資料の概要（3行以内）
2. 主要な章・セクションの構成
3. 重要ポイント（箇条書き）
4. キーワード・用語集

## 禁止事項
- 元ファイルを削除・移動しない
- 要約資料フォルダ以外への保存
```



CLAUDE.mdは「Claudeへの指示書」。ここに書けば毎回同じルールで動いてくれる。プロジェクトごとに作ると使いやすい。

CLAUDE.mdを肥大化させないために、**領域ごとにファイルを分割**する仕組み。
要約ルール・命名規則・フォーマットをそれぞれ独立したファイルで管理する。

フォルダ構成テンプレート

フォルダ構成

```
.claude/rules/
├── summarize.md  # 要約ルール
├── naming.md     # ファイル命名規則
└── format.md    # 出力フォーマット
```

記入例 — SUMMARIZE.MD（要約ルール）

summarize.md（記入例）

```
# 要約ルール

## PDFの処理
- 画像メインのPDFは pdftoppm で画像変換してから読む
- テキスト抽出できない場合はその旨を要約ファイルに明記する
- 100ページ超のPDFは章ごとに分割して読む

## 要約の粒度
- 10ページ以下：全文要約
- 10～50ページ：章ごとに要約
- 50ページ超：大見出しレベルで要約 + 重要箇所をピックアップ

## 禁止
- 著者の主張を変えて要約しない
- 数値・固有名詞は必ず原文のまま記載する
```

記入例 — NAMING.MD（命名規則）

naming.md（記入例）

```
# ファイル命名規則

## 要約ファイルのルール
- 形式：`{内容を表す日本語タイトル}_要約.md`
- スペースは使わない（アンダースコアで代替）
- 日付は不要（index.mdで管理）
```

OK例

- 差別化物販戦略塾_個人コンサルプラン要約.md
- AI_WEB_WRITING要約.md

NG例

- summary.md ← 内容が不明
- 要約 2024.md ← スペースあり

/スキル名 で呼び出せる再利用可能な手順書。

複雑な作業を1コマンドで実行できるようにする。

フォルダ構成テンプレート

フォルダ構成

```
.claude/skills/  
├── pdf-summary/  
│   └── SKILL.md
```

記入例 — SKILL.MD (PDF要約スキル)

SKILL.md (記入例)

```
---  
name: pdf-summary  
description: 指定したPDFを読み込んで要約ファイルを自動作成する  
---  
  
# PDF要約スキル  
  
## 使い方  
/pdf-summary {ファイルパスまたはフォルダパス}  
  
## 手順  
  
### Step 1: ファイル確認  
- 引数で指定されたパスのファイルを確認する  
- フォルダが指定された場合、中のPDF・MDファイルを全て処理対象にする  
  
### Step 2: テキスト抽出  
- `pdftotext` でテキスト抽出を試みる  
- テキストが少ない（スライド系）場合は `pdftoppm` で画像変換し読み込む  
  
### Step 3: 要約作成  
- `要約資料/` フォルダに要約ファイルを作成する  
- ファイル名は内容を表す日本語で命名する  
  
### Step 4: index.md更新  
- `index.md` の「作成済み要約資料」テーブルに新しい行を追加する
```

Step 5: 完了報告

- 作成したファイル名と要約の概要を1〜2文で報告する

呼び出し方

▶ ターミナルでの呼び出し例

```
/pdf-summary /Users/fushimi/Documents/新しい資料.pdf
```

特定タスク専用の「**別人格のClaude**」。専用のシステムプロンプト・ツール制限・モデルを持てる。
重い処理は軽量モデルに任せるなど、コスト最適化にも使える。

フォルダ構成テンプレート

フォルダ構成

```
.claude/agents/  
├── pdf-reader.md    # PDF読み込み専門  
├── summarizer.md    # 要約作成専門  
└── index-manager.md # index.md管理専門
```

記入例 — SUMMARIZER.MD（要約専門）

summarizer.md（記入例）

```
---  
name: summarizer  
description: 渡されたテキストを構造化して要約するエージェント。資料の要約作成に使う。  
model: claude-sonnet-4-6  
tools: Read, Write, Edit  
---  
  
# 要約専門エージェント  
  
あなたは資料を読み込んで、わかりやすく構造化する専門家です。  
  
## 行動原則  
- 読者が元資料を読まなくても内容を理解できる要約を作る  
- 図表・マトリクスは積極的にMarkdownテーブルで再現する  
- 専門用語は初出時に簡単な説明を加える  
  
## 出力フォーマット  
1. 概要（3行以内）  
2. 全体構成  
3. 章ごとの要約  
4. まとめ・重要ポイント
```

記入例 — PDF-READER.MD（ツール制限付き）

pdf-reader.md（記入例） — 軽量モデル・Bashのみ許可

```
---  
name: pdf-reader  
description: PDFからテキスト・画像を抽出する専門エージェント。Bash実行のみ許可。  
model: claude-haiku-4-5  
tools: Bash, Read  
---
```

PDF読み込み専門エージェント

できること

- pdftotext でテキスト抽出
- pdftoppm で画像変換
- ページ数・ファイルサイズの確認

できないこと

- ファイルの書き込み・編集
- 要約の作成（summarizerに渡すこと）

使われ方（メインCLAUDEからの呼び出し）

▶ 呼び出しの流れ

```
Agent(pdf-reader, "このPDFのページ数とテキストを抽出して")  
→ pdf-reader が抽出  
→ summarizer が要約作成（Write/Edit 権限あり）
```

Claudeが実行できるコマンドを許可リスト（allow）・拒否リスト（deny）で制御する。

bypassPermissions にすると確認なしで全実行が可能になる。

設定ファイル	適用範囲	Git管理
<code>~/.claude/settings.json</code>	グローバル（全プロジェクト）	しない
<code>.claude/settings.json</code>	プロジェクト共通	する
<code>.claude/settings.local.json</code>	個人ローカル設定	しない（.gitignoreへ）

記入例 — SETTINGS.JSON（資料化PJの場合）

settings.json（記入例）

```
{
  "permissions": {
    "defaultMode": "bypassPermissions",
    "allow": [
      "Bash(pdftotext *)",
      "Bash(pdftoppm *)",
      "Bash(pdftinfo *)",
      "Bash(find * -name *.pdf)",
      "Bash(mkdir *)",
      "Bash(mv *)",
      "Bash(rm -rf */slides_images)"
    ],
    "deny": [
      "Bash(rm -rf /)",
      "Bash(curl *)",
      "Bash(wget *)"
    ]
  }
}
```



bypassPermissions はすべての操作を確認なしで実行する。denyリストで危険なコマンドを必ず明示的にブロックすること。

特定のイベントに反応して**自動でコマンドを実行**する仕組み。

ファイル保存後の後処理、セッション開始時の初期化、作業完了時の通知など。

イベント	タイミング	使いどころ
SessionStart	セッション開始時	index.mdを自動読み込み・未処理ファイルを検知
PreToolUse	ツール実行前	危険なコマンドをブロック
PostToolUse	ツール実行後	ファイル保存後にindex.mdを自動更新

記入例1 — 作業完了時にMAC通知を出す

settings.json > hooks (記入例1)

```
{
  "hooks": {
    "Stop": [{
      "hooks": [{
        "type": "command",
        "command": "osascript -e 'display notification \"作業が完了しました\" with title \"Claude Code\"'"
      }]
    }]
  }
}
```

記入例2 — 要約ファイル保存後にINDEX.MDを自動更新

settings.json > hooks (記入例2)

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Write|Edit",
      "hooks": [{
        "type": "command",
        "command": "jq -r '.tool_input.file_path' | grep -q '要約資料/' && echo '要約ファイルが更新され'"
      }]
    }]
  }
}
```

```
}  
}
```

記入例3 — セッション開始時に未処理ファイルを自動検知

📄 settings.json > hooks (記入例3)

```
{  
  "hooks": {  
    "SessionStart": [{  
      "hooks": [{  
        "type": "command",  
        "command": "ls /Users/fushimi/Documents/ClaudeCode/資料化PJ/資料化してほしい/ | jq -Rs '{additi  
      }]  
    }]  
  }  
}
```

まとめ — 6要素の役割分担

① CLAUDE.md

基本行動の定義
「何をするエージェントか」
を最初に読む憲法

② Rules

細かいルールを分割管理。
CLAUDE.mdを肥大化させない

③ Skills

/コマンドで呼び出せる再利用手順
書

④ Agents

専門特化した別人格Claude（ツール・モデル制限可）

⑤ Settings

実行権限の許可・拒否リスト

⑥ Hooks

イベントに反応する自動処理



運用の鉄則： CLAUDE.mdに全部書かない。役割ごとに分割して管理することで、変更・追加がしやすくなる。